

LegalSAN: Compact Local Inference for Unfair-Clause Classification

Jiun-Yi Wu

team@derpanda.dev

Working paper · Revised preprint draft · 14 September 2026

Abstract

Can a bounded legal-language classification task be performed locally without a cloud inference service or a browser-managed generative model? We examine this question through LegalSAN, a task-specific encoder designed for application-controlled deployment. The model contains six attention-only blocks, a 12,000-entry embedding table, and 23,516,168 parameters including an eight-label classification head. Its ONNX graph integrates vocabulary-ID remapping, content masking, pooling, and classification. In a paired evaluation on 1,607 UNFAIR-ToS clauses, quantizing matrix operations and embedding lookups reduces the artifact from 94.35 MB to 24.04 MB, while eight-label macro-F1 changes from 0.7883 to 0.7870. Positive-class binary F1 (any unfair clause) is 0.8168, and 1,606 clause-level label vectors remain identical to the PyTorch reference. These observations demonstrate a compact local classification path that does not invoke cloud inference or Gemini Nano. They establish task-specific feasibility rather than superiority over generative models. Browser-wide performance, multilingual generalization, and independently held-out evaluation of the final selection policy remain to be established.

Keywords: local inference; unfair-clause detection; compact encoders; model quantization; browser deployment

1 Introduction

Online terms present a practical information problem: users must make decisions about services whose contractual conditions may be difficult to inspect. Automated clause-level prompts can help identify passages that merit attention. For an assistant operating at the point of browsing, a central systems question is where the model should execute and who controls its deployment.

Cloud inference delegates execution to a remote service. Browser-managed AI provides another route: Chrome exposes APIs for models including Gemini Nano and manages the associated model lifecycle [17]. A task-specific model distributed with an application offers a third route. It can perform a fixed prediction task using an application-selected artifact and runtime, without depending on a cloud inference endpoint or a browser-provided generative-model API.

This study asks whether such an application-controlled classifier can retain useful benchmark performance within a model artifact budget of approximately 25 MB. The task is eight-label English unfair-clause classification, not open-ended legal reasoning. We investigate a compact encoder and measure how export and quantization affect storage, output scores, and thresholded decisions.

We make three contributions: (1) a self-contained inference graph for a 23.5-million-parameter clause classifier; (2) a paired comparison of two quantization scopes that isolates the storage benefit of including embedding lookups; and (3) an analysis of decision agreement, category-level performance, and the remaining deployment requirements. The evaluated pipeline demonstrates independence from cloud and browser-managed generative inference at the model level. It does not assume that all browsers or devices provide equivalent runtime performance.

The contribution is a measured deployment case using established architecture and compression methods. Local feasibility does not establish better accuracy, speed, or generality than Gemini Nano or a cloud model; neither is evaluated here.

2 Related work and scope

BERT provides the pretrained encoder foundation [1], while LEGAL-BERT and the Contracts-BERT repository establish the legal-domain model lineage [2, 3]. UNFAIR-ToS is evaluated through LexGLUE [4]. The task itself predates this work: CLAUDETTE studied automatic detection of potentially unfair online terms [9]. Text to Trust evaluates BERT, DistilBERT, quantized adapter tuning, and prompting for unfair-term detection [13]. Its binary, rebalanced evaluation differs from the present eight-label test. Published scores under those protocols are not directly comparable to Table 1.

2.1 Attention-only encoders and architectural motivation

Our architectural motivation draws on A Controlled Study of Attention-Only Transformers [8], which examines the trade-off between feed-forward parameters and attention depth. That work studies decoder pretraining with matched-parameter, matched-compute, and matched-depth comparisons. LegalSAN instead uses a compact bidirectional encoder for supervised clause classification and examines its export. This extends the application context from decoder pretraining to task-specific encoder deployment; the effectiveness of the architectural choice is not isolated by the export experiment.

2.2 Vocabulary compression and combined pipelines

DistilBERT is an established example of encoder distillation [5]. Fast Vocabulary Transfer [10] combines domain-specific vocabulary adaptation with compression and evaluates LEDGAR, among other tasks. It learns a new tokenizer and transfers embeddings; the LegalSAN export retains original segmentation and remaps discarded IDs to an unknown embedding. Load What You Need [15] also reduces embedding vocabulary to obtain smaller multilingual encoders. Thus, identifying vocabulary storage as a compression target is established practice.

EI-BERT [12] combines hard token pruning, cross-distillation, and module-wise post-training quantization for edge NLU. It is a close precedent for combining these compression ingredients and reports a 1.91 MB model in different tasks and deployment settings. Our study focuses on six attention-only blocks and clause-level export, with a different evaluation objective. The reported model sizes do not provide a controlled performance comparison across tasks.

2.3 Quantization and local inference

Q8BERT [11] studies eight-bit BERT compression through quantization-aware fine-tuning. The present experiment instead uses ONNX Runtime dynamic post-training quantization [6] and compares two operator scopes. WebLLM [14] establishes browser-local language-model inference as an existing systems direction; ONNX Runtime Web [7] supplies the intended runtime here. Polisis [16] is an earlier neural system for privacy-policy analysis. These systems provide context for the present emphasis on a compact, application-controlled classifier for a fixed legal taxonomy.

2.4 Contribution boundary

This paper contributes a checkpoint-specific engineering evaluation: a custom encoder inference graph, a MatMul-versus-MatMul-plus-Gather quantization comparison, and paired numerical and label-decision checks on UNFAIR-ToS. The unit of analysis is one fixed checkpoint, `legalsan_finetuned.pt`, before and after export. This permits paired measurement of conversion effects while leaving comparisons between training recipes to future work. The full training provenance and evidence boundary are addressed in Section 7.

3 Model and inference graph

3.1 Compact checkpoint

Property	Recorded configuration
Encoder blocks	6; feed-forward sublayers absent
Hidden width / attention heads	768 / 12; 12 key-value groups
Vocabulary / positions	12,000 retained embeddings / 128 positions
Normalization	Post-attention LayerNorm; retained output LayerNorm
Task output	Mean pooling followed by a linear 8-label head
Total parameters	23,516,168, including classification head
Decision threshold	0.40 for every label

The encoder uses learned position and token-type embeddings. Each block applies multi-head self-attention with a residual connection and normalization, followed by the retained output normalization. It contains no feed-forward network. Evaluation mode disables dropout. Padded key positions receive an additive attention bias of -10,000 in the export implementation.

3.2 Token mapping and pooled representation

Tokenization uses the original 30,522-entry Contracts-BERT vocabulary. A lookup buffer maps its IDs to the retained 12,000-row table; dropped IDs map to the compact unknown-token ID. This preserves the original segmentation rather than re-tokenizing against a smaller vocabulary. It also means that discarded pieces lose lexical identity. The export study does not quantify coverage loss or performance by out-of-vocabulary rate.

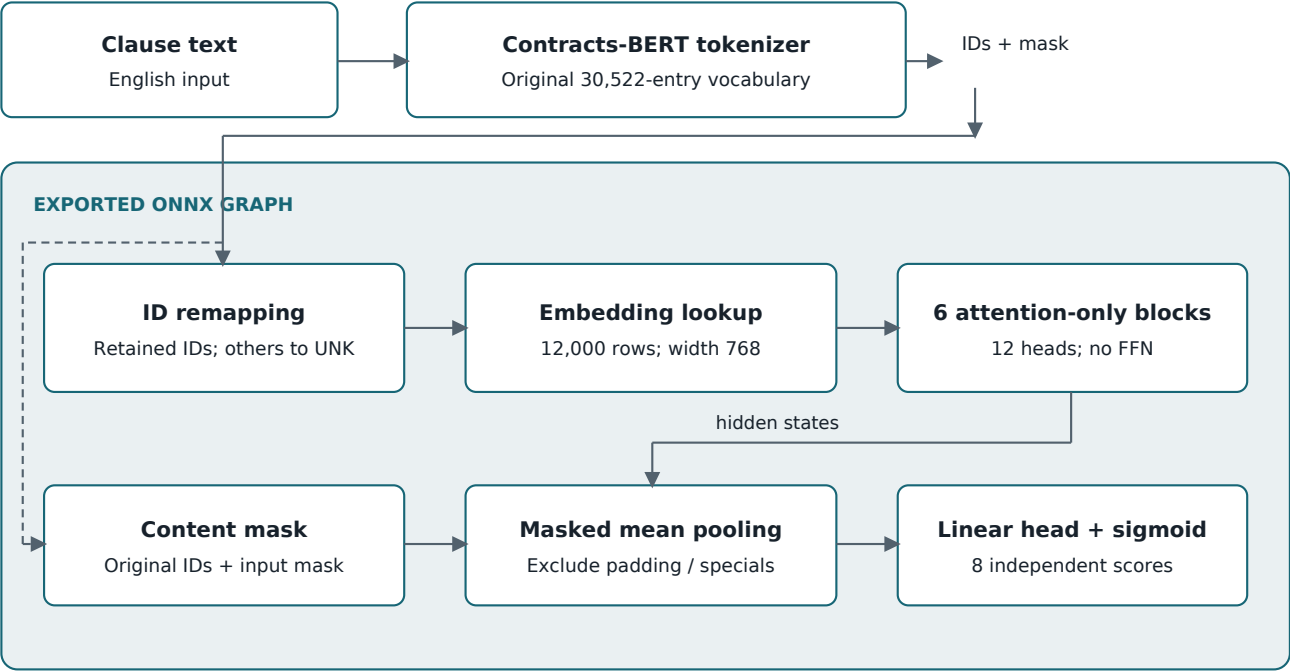
The pooling mask excludes padding and original special-token IDs {0, 100, 101, 102, 103}. A dropped ordinary token remains in the pooling mask even when its embedding is remapped to the unknown row. For final hidden states h and binary content mask m , the graph computes:

$$z = (\sum_t m_t h_t) / \max(1, \sum_t m_t), \quad p = \text{sigmoid}(Wz + b).$$

There is no L2 normalization of the pooled vector in this export path. Each label is predicted when its sigmoid output is at least 0.40. The eight outputs are independent scores, not a mutually exclusive distribution.

A. Clause-to-score inference path

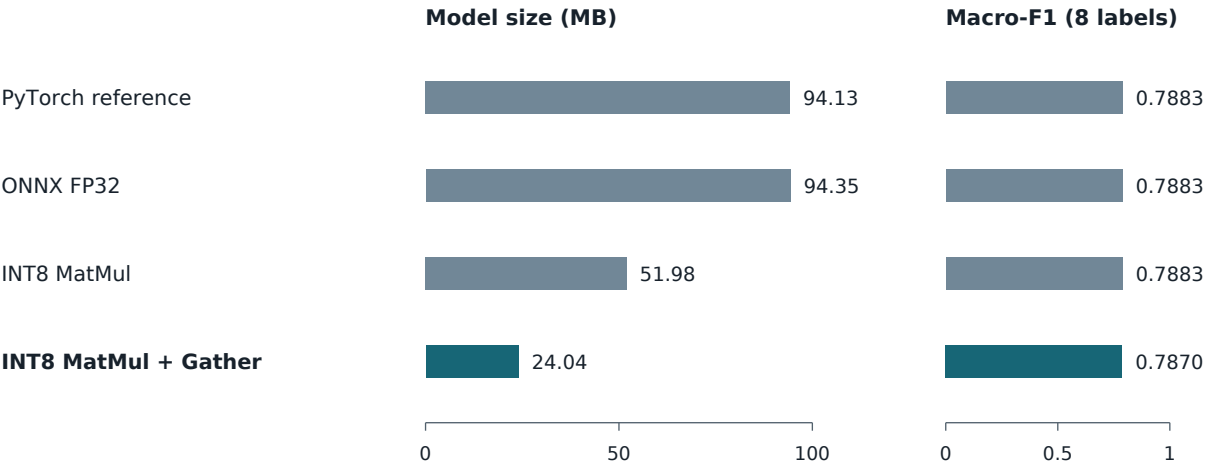
APPLICATION / JAVASCRIPT



Application applies threshold 0.40 to each output; browser timing is a separate validation step.

Figure 1. The application tokenizes clause text; the ONNX graph handles remapping, embeddings, attention, content masking, pooling, and classification. The dashed branch supplies original IDs and the input mask to the content-mask operation. The input attention mask also masks padded keys inside attention (connection omitted for clarity). Runtime binaries and tokenization remain outside the model artifact.

B. Artifact size and classification performance



FP32 ONNX to full INT8: 74.5% smaller; one of 1,607 clause-level label vectors changes.

Figure 2. Four versions of the same checkpoint, evaluated on the same 1,607 clauses at threshold 0.40. Both axes begin at zero. Labels preserve the small score difference that is not visually distinguishable at this scale. Model size excludes runtime assets.

4 Export and evaluation protocol

4.1 Export and quantization

Figure 1 summarizes the inference boundary. The wrapper accepts original token IDs and attention masks as INT64 tensors. ID remapping, content masking, pooling, the classification head, and sigmoid are embedded in the graph. Export uses ONNX opset 17 with dynamic batch and sequence axes. The learned position table still limits supported sequence length to 128 tokens. Structural validation uses the ONNX checker.

Two variants use ONNX Runtime dynamic quantization with signed INT8 weights, per-channel quantization enabled, and `MatMulConstBOnly=True`. The first targets `MatMul`; the second targets `MatMul` and `Gather`, thereby including eligible embedding lookups. Preprocessing succeeded in the recorded run. INT8 describes the quantization path, not a claim that every operation, activation, or graph value uses eight-bit arithmetic.

4.2 Data and metrics

The notebook loads UNFAIR-ToS from `coastalcph/lex_glue`, with 5,532 training, 2,275 validation, and 1,607 test examples. The export experiment evaluates the test split only. Whitespace is normalized; tokenization truncates and pads to 128 tokens. Inference uses batches of 32 and the CPU execution provider. The threshold of 0.40 is loaded from the checkpoint; its upstream selection history is not established by this notebook.

For each of eight labels, $F1 = 2TP / (2TP + FP + FN)$, with zero assigned for a zero denominator. Macro-F1 averages these eight scores. Micro-F1 aggregates counts over labels. Binary F1 collapses all labels into an any-unfair decision and reports F1 for the unfair positive class; it does not average fair-class and unfair-class F1. Exact match compares the complete predicted vector with the ground truth; decision agreement instead compares each exported vector with the PyTorch vector. No ninth fair-class F1 is included.

The same checkpoint is evaluated before and after export, making drift comparisons paired. The selected quantization variant, however, is chosen using test macro-F1: the script falls back to `MatMul`-only if adding `Gather` loses more than 0.005. These are exploratory test results, not an untouched final assessment of a fully prespecified deployment policy.

5 Results

Table 1. Recorded artifact sizes and test performance. MB denotes 10^6 bytes. Scores are rounded to four decimals.

Variant	MB	Macro-F1	Micro-F1	Binary F1	Exact match
PyTorch reference	94.13	0.7883	0.7619	0.8193	0.9527
ONNX FP32	94.35	0.7883	0.7619	0.8193	0.9527
INT8 MatMul	51.98	0.7883	0.7619	0.8193	0.9527
INT8 MatMul + Gather	24.04	0.7870	0.7598	0.8168	0.9521

Figure 2 visualizes the paired results. Relative to FP32 ONNX, `MatMul`-only quantization reduces size by 44.9%. Including `Gather` reduces size by 74.5%, or approximately 3.93 times, and saves a further 27.94 MB over `MatMul`-only. The $12,000 \times 768$ word-embedding table alone contains 9.216 million weights, or 36.864 MB in FP32. This explains why quantizing linear operations without embedding lookups leaves a substantial storage cost.

The notebook reports a macro-F1 change of -0.0012 from the unrounded values. Subtracting the independently rounded Table 1 endpoints gives -0.0013; this is a rounding difference. No confidence interval or multi-checkpoint quantization replication is available.

5.1 Numerical and decision consistency

Table 2. Quantization drift across all 1,607 clauses, relative to the same PyTorch checkpoint.

Variant	Maximum absolute score difference	Whole-clause decision agreement
ONNX FP32	0.0000 (rounded)	100%
INT8 MatMul	0.0878	100%
INT8 MatMul + Gather	0.1821	99.94%

The integrated wrapper matches the separate remap-and-pool implementation exactly on five example clauses. FP32 ONNX differs from the wrapper by at most 8.39×10^{-8} on those examples; a separate short-sequence check at length 16 yields 8.42×10^{-8} . These checks establish agreement for the tested inputs, not a proof over all possible sequences.

The rounded agreement of 0.9994 on 1,607 clauses corresponds to 1,606 identical label vectors. This does not specify how many individual labels changed within the remaining clause. The large maximum score difference also shows why decision stability at one threshold must not be interpreted as probability calibration or stability at arbitrary thresholds.

5.2 Category-level performance

Table 3. INT8 MatMul + Gather performance. Support counts positive examples per label; multi-label supports need not sum to the number of clauses.

Category	Precision	Recall	F1	Support
Limitation of liability	0.758	0.658	0.704	38
Unilateral termination	0.744	0.763	0.753	38
Unilateral change	0.750	0.632	0.686	38
Content removal	0.625	0.769	0.690	13
Contract by using	0.941	0.696	0.800	23
Choice of law	1.000	0.923	0.960	13
Jurisdiction	0.933	0.875	0.903	16
Arbitration	0.750	0.857	0.800	7

Seven categories achieve observed precision of at least 0.70. The notebook recommends withholding Content removal from the interface. Table 1 nevertheless evaluates all eight categories; it is not a measurement of a seven-category interface. Choosing categories from test precision is another test-informed decision requiring subsequent independent evaluation.

Rare-category estimates are fragile. Arbitration has seven positive examples; its displayed precision and recall correspond to six true positives, two false positives, and one false negative. Choice of law has observed precision of 1.000, but only 13 positive examples. Neither observation warrants a broad reliability guarantee.

5.3 Relationship to website headline metrics

The project landing page is reported to display 95.88% overall accuracy and 88.46% binary macro-F1 from a prior LegalBench run. Table 4 records these figures to distinguish that reported evaluation from the UNFAIR-ToS export experiment. The website values and their attribution to LegalBench were supplied in manuscript feedback; the original evaluation script, predictions, and run metadata are not part of the experimental record examined here. They are contextual reports, not additional measurements reproduced in this paper.

Table 4. Evaluation provenance and metric definitions. Website figures are reported values; notebook figures refer to the INT8 MatMul + Gather variant.

Evaluation / evidence	Metric	Value
Prior LegalBench run; website figures reported in feedback	Overall accuracy; exact task and scoring rule unverified	95.88%
Prior LegalBench run; website figures reported in feedback	Binary macro-F1; original calculation unverified	88.46%
UNFAIR-ToS; 1,607 clauses; saved notebook outputs	Eight-label macro-F1	0.7870 (78.70%)
UNFAIR-ToS; same recorded evaluation	Positive-class binary F1 (any unfair)	0.8168 (81.68%)
UNFAIR-ToS; same recorded evaluation	Eight-label exact-match accuracy	0.9521 (95.21%)

These metric names specify different aggregation rules. Under the standard definition, binary macro-F1 = $(F1_{\text{fair}} + F1_{\text{unfair}}) / 2$. In contrast, the notebook binary F1 is $F1_{\text{unfair}}$ after reducing each eight-label vector to whether any unfair label is present. Its eight-label macro-F1 averages the F1 scores of the eight unfairness categories and excludes an additional fair-class score. Consequently, the reported 88.46% binary macro-F1 is not the same quantity as either 81.68% positive-class binary F1 or 78.70% eight-label macro-F1.

Likewise, the paper reports exact-match accuracy: a clause counts as correct only when its entire eight-label vector matches the annotation. The website label “overall accuracy” does not establish whether the prior run used binary accuracy, exact match, or another aggregation. Its 95.88% must therefore not be relabeled as the paper’s eight-label exact match of 95.21%. Converting proportions to percentages changes presentation only; it does not make the evaluations equivalent.

No improvement or degradation between the website and paper can be inferred from these numbers. A valid comparison requires the prior LegalBench task or subset, data revision and split, sample count, label mapping, checkpoint and export variant, preprocessing, threshold, and metric implementation, preferably with per-example predictions. Those details would allow the two evaluations to be reproduced and, where appropriate, rescored under a common protocol. Until then, the website figures remain a separately reported prior evaluation and do not support this paper’s measured conclusions. Public presentation should identify each figure’s evaluation source and full metric name, and should not imply that this paper validates the prior LegalBench scores.

6 Deployment boundary

The export emits an ONNX model, a 0.23 MB vocabulary, a configuration file, and a 0.08 MB tokenizer golden file. The latter contains 200 sampled test clauses plus six edge cases covering accents, punctuation, mixed alphanumeric text, whitespace, long words, and Chinese characters. These are reference token sequences for checking a JavaScript implementation. Generating them is not itself evidence that the JavaScript implementation passes.

6.1 Runtime evidence and remaining browser validation

The recorded native CPU timing is 7.4 ms per clause, averaged over 50 calls after five warm-up calls. It uses one fixed clause padded to 128 tokens and excludes tokenization and session initialization. CPU model and thread count are not recorded; the session uses default threading settings. This measurement is therefore a diagnostic observation, not a reproducible browser benchmark or a guaranteed lower bound on browser latency.

ONNX Runtime Web is the intended browser execution environment [7]. Preliminary project notes report a 49 ms WASM observation, tokenizer parity checks, and a live-page trial. Because the associated runtime harness and execution logs are not included in the experimental record, this paper bases its performance conclusions on the recorded native evaluation. A browser study should record device, browser and runtime versions, sequence lengths, warm-up, cold start, latency distribution, and peak memory.

Dynamic shapes support shorter inputs and variable batches, but larger batches are not assumed to be faster on every browser. The deployed package also includes runtime binaries and tokenizer assets, so 24.04 MB is the model artifact size, not the complete download or memory footprint. Long clauses require a documented segmentation or windowing policy rather than silent truncation.

7 Discussion and limitations

7.1 A task-specific alternative to generative inference

The central finding is that the evaluated classification operation can be performed by a compact encoder without calling a generative model. The 24.04 MB artifact produces eight task-specific scores locally. Its macro-F1 of 0.7870 and binary F1 of 0.8168 provide evidence that this is a substantive prediction path rather than only a demonstration of model loading. Practical usefulness still depends on the tolerated false-positive and false-negative rates in the intended interface.

The absence of cloud or Gemini Nano calls is a property of the implemented inference path; performance on UNFAIR-ToS supplies the complementary task evidence. This supports a claim of task-specific feasibility. It does not establish equivalence to a particular generative model, whose prompts, version, and output mapping would need to be controlled in a direct comparison. Training and artifact distribution are separate from inference: the present result does not imply that model development was cloud-free.

Application-controlled delivery also permits the developer to select and version the model artifact alongside the inference code. With required assets provisioned locally, classification need not transmit clause text to a remote inference service. The complete extension would still require inspection of telemetry, logging, and network behavior before making an end-to-end privacy claim. Model download and runtime maintenance also remain operational costs.

7.2 Accuracy, uncertainty, and decision stability

Quantization preserves nearly all decisions at threshold 0.40, but the maximum absolute score change of 0.1821 shows that score stability is a stronger requirement. The evidence supports fixed-threshold classification more directly than confidence display or score-based ranking. Calibration, threshold sensitivity, and analysis of the changed clause are therefore important follow-up experiments.

Macro-F1, positive-class binary F1, and exact match answer different questions. Section 5.3 separately accounts for the reported website headline metrics and their unresolved evaluation provenance. The exact-match value of 0.9521 measures full-vector agreement with ground truth and should not be substituted for the ability to detect rare unfair categories. Category-level results in Table 3 make the precision-recall trade-off explicit. Small positive supports preclude broad reliability claims for rare labels.

7.3 Threats to validity and next experiments

The export study uses a single checkpoint and one English benchmark. Quantization scope and the recommended displayed categories were selected using test results, so the deployed policy requires assessment on new, independently held-out documents. Full upstream fine-tuning records, model-selection history, and repeated-seed measurements are not available in the experimental record used here. The paired export comparison remains informative about this checkpoint, but it does not estimate variability across trained models.

Native CPU execution verifies the graph on the measured platform. It does not establish browser-wide latency, memory use, or tokenizer parity. A full evaluation should measure cold start, steady-state latency distributions, peak memory, tokenization, and document processing on specified devices and browsers, including short sequences and variable batches. The 128-token limit requires explicit treatment of long clauses.

German AGB, cross-clause reasoning, and open-ended legal explanation are outside the present task. Classification indicates similarity to benchmark annotations, not a legal determination. Future experiments should compare equally constrained compact encoders, independently evaluate final display rules, and then test multilingual and real-site generalization. A generative-model baseline would answer a separate question about relative quality and cost.

8 Conclusion

LegalSAN demonstrates a compact local inference path for English unfair-clause classification without invoking cloud inference or a browser-managed generative model such as Gemini Nano. Joint MatMul and Gather quantization yields a 24.04 MB artifact with eight-label macro-F1 of 0.7870 and binary F1 of 0.8168, preserving the PyTorch label vector for 1,606 of 1,607 test clauses. The contribution is evidence that a bounded consumer-facing language task can be implemented with an application-controlled encoder. Independent policy evaluation and complete browser measurements are the next steps toward establishing practical deployment suitability.

Data, code, and reproducibility

The primary experimental record is `export-onnx-browser.ipynb`: checkpoint configuration and parameter count (cells 3-4), wrapper checks (cell 6), export checks (cell 8), quantization sizes (cell 10), full-split results (cell 12), and per-label metrics and timing (cell 13). Cell indices are zero-based. Results in this paper are transcribed from saved outputs; the original checkpoint and exported binary were not independently rerun during manuscript preparation. No public code or checkpoint release is claimed in this version.

The recorded runtime is ONNX Runtime 1.29.0; installation requests Transformers 4.57.6 and version ranges for ONNX, datasets, and other dependencies. The export notebook does not pin dataset or tokenizer revisions, provide a complete resolved environment, or record checkpoint hashes. A reproducible release should supply those identifiers, the upstream fine-tuning recipe, checkpoint-selection history, model binaries, and per-example predictions. This first version makes export-stage methods inspectable but does not claim end-to-end training reproducibility.

Source notebook SHA-256:
2b53ef2fe037b175b6fd2f61d6eee8a6
4bc69cc4ba16b693cfd7e7861ba0b47c

AI assistance. OpenAI ChatGPT assisted with manuscript drafting, code inspection, and document preparation. It did not execute new training or browser experiments for this manuscript.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT, pp. 4171-4186. [Source](#).
- [2] Ilias Chalkidis et al. 2020. LEGAL-BERT: The Muppets straight out of Law School. Findings of EMNLP. [Source](#).
- [3] NLP AUJEB. bert-base-uncased-contracts. Model repository and model card. Accessed 10 September 2026. [Source](#).
- [4] Ilias Chalkidis et al. 2022. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. ACL, pp. 4310-4330. [Source](#).
- [5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. [Source](#).
- [6] ONNX Runtime contributors. Quantize ONNX models. Documentation. Accessed 10 September 2026. [Source](#).
- [7] ONNX Runtime contributors. Web. Documentation. Accessed 10 September 2026. [Source](#).
- [8] Henry Ndubuaku et al. 2026. A Controlled Study of Attention-Only Transformers. arXiv:2607.18363. [Source](#).
- [9] Marco Lippi et al. 2018. CLAUDETTE: an Automated Detector of Potentially Unfair Clauses in Online Terms of Service. arXiv:1805.01217. [Source](#).
- [10] Leonidas Gee, Andrea Zugarini, Leonardo Rigutini, and Paolo Torroni. 2022. Fast Vocabulary Transfer for Language Model Compression. EMNLP Industry Track, pp. 409-416. [Source](#).
- [11] Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8BERT: Quantized 8Bit BERT. arXiv:1910.06188. [Source](#).
- [12] Maolin Wang et al. 2025. Put Teacher in Student's Shoes: Cross-Distillation for Ultra-compact Model Compression Framework. KDD 2025. doi:10.1145/3711896.3737257. [Source](#).
- [13] Noshitha Padma Pratyusha Juttu et al. 2025. Text to Trust: Evaluating Fine-Tuning and LoRA Trade-offs in Language Models for Unfair Terms of Service Detection. arXiv:2510.22531. [Source](#).
- [14] Charlie F. Ruan et al. 2024. WebLLM: A High-Performance In-Browser LLM Inference Engine. arXiv:2412.15803; revised 2026. [Source](#).
- [15] Amine Abdaoui, Camille Pradel, and Gregoire Sigel. 2020. Load What You Need: Smaller Versions of Multilingual BERT. SustainLP, pp. 119-123. [Source](#).
- [16] Hamza Harkous et al. 2018. Polisis: Automated Analysis and Presentation of Privacy Policies Using Deep Learning. USENIX Security 2018. [Source](#).
- [17] Google. Built-in AI. Chrome for Developers. Accessed 14 September 2026. [Source](#).